

Collegium Witelona Uczelnia Państwowa w Legnicy
Wydział Nauk Technicznych i Ekonomicznych
Kierunek: Informatyka



Podstawy symulacji komputerowej"

Temat: "Opracowanie zadań projektowych".

Autorzy

Mateusz Bogacz-Drewniak, nr. indeksu: 44491

Robert Kuźba, nr. indeksu: 38575

Prowadzący przedmiot
dr inż. Ryszard Rębowski

Legnica, 2026

Spis treści

1	Lista 0 – zadanie 2	2
1.1	Treść zadania	2
1.2	Rozwiązanie analityczne	2
1.2.1	Metoda 1: Bezpośrednie sumowanie prawdopodobieństw	2
1.2.2	Metoda 2: Sumowanie z definicji podanej w zadaniu	2
1.3	Implementacja	3
1.4	Odpowiedź końcowa	4
2	Lista 1 - zadanie 2	5
2.1	Treść zadania	5
2.2	Opis metody	5
2.3	Implementacja	5
2.4	Wyniki symulacji i testy graficzne	6
2.5	Interpretacja wyników	8
3	Lista 2 - zadanie 5	8
3.1	Treść zadania	8
3.2	Model probabilistyczny	9
3.3	Metoda zdarzeń dyskretnych	10
3.4	Implementacja	11
3.5	Wyniki symulacji	11
3.6	Odpowiedź końcowa	12

1 Lista 0 – zadanie 2

1.1 Treść zadania

Wiadomo, że dla zmiennej losowej X mamy:

$$X(\Omega) = \{1, 2, 3, 4\}$$

$$P(\{\omega \in \Omega : X(\omega) = i\}) = ic, \quad \text{dla pewnej } c \in R.$$

Obliczyć dwoma metodami $P(\{\omega \in \Omega : 2 \leq X(\omega) \leq 3\})$.

1.2 Rozwiązanie analityczne

Z własności rozkładu prawdopodobieństwa (warunek unormowania) wiemy, że suma prawdopodobieństw wszystkich zdarzeń elementarnych musi wynosić 1. Zatem:

$$\sum_{i=1}^4 P(X = i) = 1$$

$$1c + 2c + 3c + 4c = 1$$

$$10c = 1 \implies c = \frac{1}{10}.$$

Mając stałą c , rozkład zmiennej losowej X prezentuje się następująco: $P(X = 1) = 0,1$, $P(X = 2) = 0,2$, $P(X = 3) = 0,3$, $P(X = 4) = 0,4$.

1.2.1 Metoda 1: Bezpośrednie sumowanie prawdopodobieństw

Z racji, że interesują nas zdarzenia dla $X = 2$ oraz $X = 3$, dodajemy bezpośrednio ich prawdopodobieństwa z wyznaczonego rozkładu:

$$P(2 \leq X \leq 3) = P(X = 2) + P(X = 3) = 0,2 + 0,3 = 0,5.$$

1.2.2 Metoda 2: Sumowanie z definicji podanej w zadaniu

Wykorzystujemy podany wzór $P(X = i) = i \cdot c$:

$$P(2 \leq X \leq 3) = \sum_{i=2}^3 i \cdot c = 2c + 3c = 5c.$$

Podstawiając $c = \frac{1}{10}$, otrzymujemy:

$$5 \cdot \frac{1}{10} = \frac{5}{10} = 0,5.$$

1.3 Implementacja

Poniżej przedstawiono kod w języku Python symulujący rozwiązanie tego zadania wraz z generacją wykresów rozkładu zmiennej losowej i jej dystrybuanty.

```
import matplotlib.pyplot as plt
import numpy as np

print("\n1. Znalezienie stalej c:")
print("X( ) = {1, 2, 3, 4}")
print("P(X = i) = i * c")
print("\nWarunek: suma wszystkich prawdopodobieństw = 1")
print("P(X=1) + P(X=2) + P(X=3) + P(X=4) = 1")
print("1 * c + 2 * c + 3 * c + 4 * c = 1")
print("10 * c = 1")

c = 1/10
print(f"\033[1mStala c:\033[0m {c} \n")
values = [1, 2, 3, 4]
probabilities = [i * c for i in values]

print(f"P(X=1) = 1 * {c} = {probabilities[0]}")
print(f"P(X=2) = 2 * {c} = {probabilities[1]}")
print(f"P(X=3) = 3 * {c} = {probabilities[2]}")
print(f"P(X=4) = 4 * {c} = {probabilities[3]} \n")
print(f"\033[1mSuma:\033[0m {sum(probabilities)} \n")

print("Metoda 1: Bezposrednie sumowanie")
print("P(2 <= X <= 3) = P(X=2) + P(X=3)")
print(f"= 2 * {c} + 3 * {c}")
print(f"= 2 * {c} + 3 * {c}")
print(f"= {2*c} + {3*c}")

result_method1 = probabilities[1] + probabilities[2]
print(f"\033[1m= {result_method1} \033[0m\n")

print("Metoda 2: Przez sumowanie zdan 2 i 3 z definicji")
print("P(2 <= X <= 3) = P(X=i) dla i {2, 3}")

result_method2 = sum(probabilities[i-1] for i in [2, 3])
print(f"= {probabilities[1]} + {probabilities[2]}")

print(f"\033[1m= {result_method2} \033[0m\n")

print("Weryfikacja zgodnosci wynikow:")
print(f"\033[1mMetoda 1: {result_method1}\033[0m")
print(f"\033[1mMetoda 2: {result_method2}\033[0m")
print(f"\033[1mZgodnosc: {'Tak' if abs(result_method1 - result_method2) < 1e-10 else 'Nie'}\033[0m\n")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ax1 = axes[0]
colors = ['lightblue', 'salmon', 'salmon', 'lightblue']
bars = ax1.bar(values, probabilities, color=colors, edgecolor='black', linewidth=2, width=0.6)

bars[1].set_color('salmon')
bars[2].set_color('salmon')

ax1.set_xlabel('X', fontsize=12, fontweight='bold')
ax1.set_ylabel('P(X = i)', fontsize=12, fontweight='bold')
ax1.set_title('Rozklad zmiennej losowej X', fontsize=13, fontweight='bold')
ax1.set_xticks(values)
ax1.grid(axis='y', alpha=0.3, linestyle='--')
ax1.set_ylim([0, max(probabilities) * 1.2])

for i, (val, prob) in enumerate(zip(values, probabilities)):
    ax1.text(val, prob + 0.01, f'{prob:.1f}', ha='center', fontsize=11, fontweight='bold')

ax1.text(0.5, 0.95, 'Kolor jasnoniebieski: nie wliczone do P(2 <= X <= 3)',
        transform=ax1.transAxes, fontsize=10, verticalalignment='top',
```

```

        bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.5))
ax1.text(0.5, 0.85, 'Kolor lososowy: wliczone do P(2      X      3)',
        transform=ax1.transAxes, fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='salmon', alpha=0.5))

ax2 = axes[1]
cumulative_probs = np.cumsum(probabilities)
x_cdf = [0] + values
y_cdf = [0] + list(cumulative_probs)

ax2.step(x_cdf, y_cdf, 'b-', linewidth=2, marker='o', markersize=8, where='post', label='
    F(x) = P(X      x)')
ax2.fill_between([1.99, 3.01], 0, 1, alpha=0.3, color='salmon', label='P(2      X      3)')

ax2.set_xlabel('X', fontsize=12, fontweight='bold')
ax2.set_ylabel('F(X)', fontsize=12, fontweight='bold')
ax2.set_title('Dystrybuanta zmiennej losowej X', fontsize=13, fontweight='bold')
ax2.grid(True, alpha=0.3, linestyle='--')
ax2.set_xlim([-0.5, 5])
ax2.set_ylim([-0.05, 1.1])
ax2.legend(fontsize=10)

for i, cum_prob in enumerate(cumulative_probs):
    ax2.text(x_cdf[i], cum_prob + 0.05, f'{cum_prob:.2f}', ha='center', fontsize=9)

plt.tight_layout()
print("\nWizualizacja zapisana do: zad2_lista0_visualization.png \n")
plt.show()

print(f"\033[1mPodsumowanie:\033[0m")
print(f"P(2      X      3) = {result_method1} = {result_method1.as_integer_ratio()[0]}/{
    result_method1.as_integer_ratio()[1]} = 50%")

```

Listing 1: Implementacja dwóch rozwiązań zadania

1.4 Odpowiedź końcowa

Obydwie metody analityczne, jak i weryfikacja numeryczna za pomocą skryptu Python, doprowadziły do tego samego wyniku. Szukane prawdopodobieństwo wynosi:

$$P(2 \leq X \leq 3) = 0,5$$

czyli:

$$50\%.$$

2 Lista 1 - zadanie 2

2.1 Treść zadania

Napisać program wykonujący algorytm kwadratowy von Neumanna.

2.2 Opis metody

Algorytm środka kwadratu (zaproponowany przez Johna von Neumanna) to jedna z najstarszych metod generowania liczb pseudolosowych. Metoda opiera się na następujących krokach:

1. Wybór początkowej wartości ziarna (ang. *seed*) o długości n cyfr.
2. Podniesienie tej wartości do kwadratu, co daje liczbę o długości do $2n$ cyfr.
3. Uzupełnienie wyniku zerami wiodącymi (jeśli to konieczne), aby zachować stałą długość $2n$.
4. Wyciągnięcie n środkowych cyfr, które stają się kolejną wygenerowaną liczbą i nowym ziarnem dla następnej iteracji.

Częstym problemem tego algorytmu jest szybkie zapętlenie się lub zbieganie do zera. W poniższej implementacji wprowadzono mechanizm zapobiegający temu zjawisku poprzez resetowanie wartości w przypadku osiągnięcia zera.

2.3 Implementacja

Symulację wykonano w języku Python. Poniżej przedstawiono implementację generatora oraz funkcji wizualizujących niezależność wygenerowanych próbek w przestrzeni 2D i 3D:

```
import matplotlib.pyplot as plt

def von_neumann_generator(seed, n_samples):
    current = seed
    results = []

    for i in range(n_samples):
        # Podniesienie do kwadratu i wyrownanie do 8 znakow
        square_str = str(current**2).zfill(8)

        # Pobranie 4 srodkowych cyfr (indeksy 2,3,4,5)
        middle_str = square_str[2:6]
        current = int(middle_str)

        # Normalizacja do przedzialu [0, 1)
        results.append(current / 10000.0)

        # Zabezpieczenie przed utknieciem w zerze
        if current == 0:
            current = 1234 + i

    return results

def plot_2d_independence(data):
    x = data[:-1]
    y = data[1:]

    plt.figure(figsize=(8, 8))
    plt.scatter(x, y, alpha=0.6, s=15, color='blue')
    plt.title("Test graficzny 2D: Von Neumann ($r_n$ vs $r_{n+1}$")
```

```

plt.xlabel("$r_n$")
plt.ylabel("$r_{n+1}$")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

def plot_3d_independence(data):
    # Tworzymy trojki (x, y, z)
    x = data[:-2]
    y = data[1:-1]
    z = data[2:]

    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    ax.scatter(x, y, z, alpha=0.6, s=10, color='purple')
    ax.set_title("Test graficzny 3D: Von Neumann ($r_n, r_{n+1}, r_{n+2}$)")
    ax.set_xlabel("$r_n$")
    ax.set_ylabel("$r_{n+1}$")
    ax.set_zlabel("$r_{n+2}$")
    plt.show()

# Wykonanie algorytmu
samples = von_neumann_generator(seed=1234, n_samples=500)

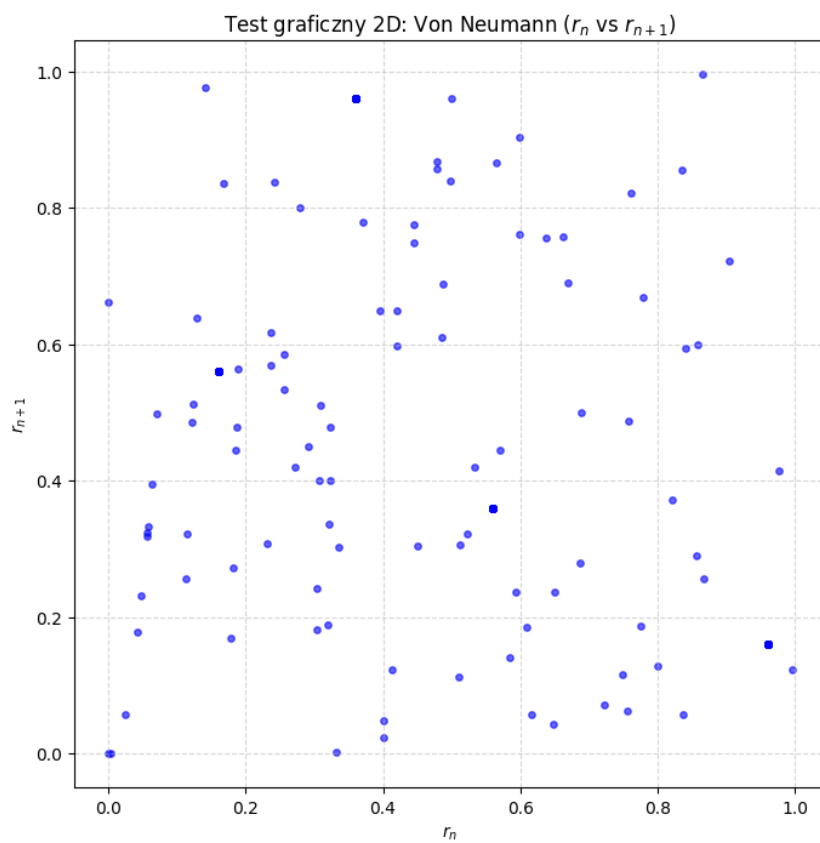
```

Listing 2: Implementacja algorytmu kwadratowego von Neumanna

2.4 Wyniki symulacji i testy graficzne

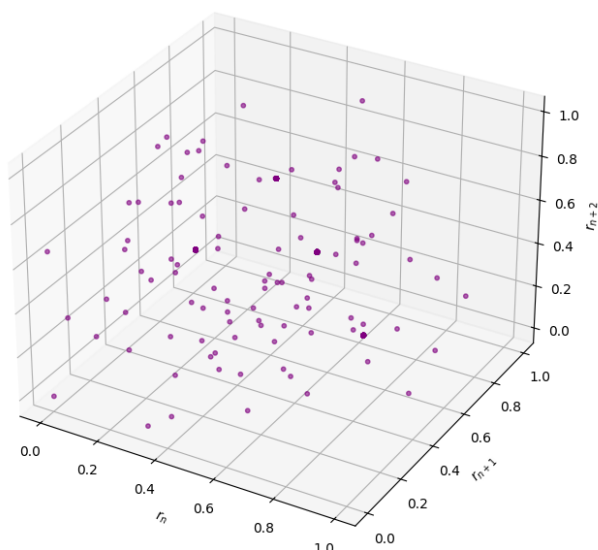
Wygenerowano próbkę $N = 500$ liczb pseudolosowych dla początkowego ziarna 1234. Pierwsze wartości wygenerowanego ciągu to m.in.: 0.5227, 0.3215, 0.0336, 0.1128.

Aby sprawdzić jakość wygenerowanego ciągu i niezależność kolejnych próbek, przeprowadzono testy graficzne w wymiarze 2D (punkty o współrzędnych r_n, r_{n+1}) oraz 3D (punkty o współrzędnych r_n, r_{n+1}, r_{n+2}).



Rysunek 1: Rozkład par (r_n, r_{n+1}) w teście 2D.

Test graficzny 3D: Von Neumann (r_n, r_{n+1}, r_{n+2})



Rysunek 2: Rozkład trójek (r_n, r_{n+1}, r_{n+2}) w teście 3D.

2.5 Interpretacja wyników

Na podstawie przeprowadzonych testów graficznych (wykresy 2D i 3D) można wstępnie ocenić jakość generatora. Zbyt wyraźne wzory, prążki lub skupiska punktów na wykresach świadczyłyby o silnej zależności między kolejnymi próbkami, co jest charakterystyczną wadą prostych generatorów (takich jak klasyczny algorytm von Neumanna). Zastosowane w kodzie zabezpieczenie (zmiana stanu przy wygenerowaniu 0) zapobiega całkowitej degradacji ciągu, jednak dla profesjonalnych zastosowań symulacyjnych metoda ta ustępuje nowoczesnym generatorom.

3 Lista 2 - zadanie 5

3.1 Treść zadania

Zakładamy, że szkody klientów zgłaszane są według procesu Poissona o intensywności 10 na dzień. Kwota szkody jest zmienną losową o rozkładzie wykładniczym ze średnią \$1000. Firma ubezpieczeniowa otrzymuje wpłaty w sposób ciągły ze stałą intensywnością \$11000 na dzień. Przyjmując kapitał początkowy \$25000, oszacować metodą symulacji prawdopodobieństwo, że kapitał pozostanie dodatni przez pierwsze 365 dni.

3.2 Model probabilistyczny

Niech $U(t)$ oznacza kapitał firmy w chwili t , gdzie $t \in [0, 365]$. Przyjmujemy:

$$U(0) = 25000.$$

Wpłaty napływają ciągle ze stałą intensywnością:

$$c = 11000 \quad [\text{\$ na dzień}].$$

Między kolejnymi szkodami kapitał zmienia się liniowo:

$$\frac{dU}{dt} = c,$$

o ile nie wystąpi zdarzenie szkody.

Zgłoszenia szkód tworzą proces Poissona o intensywności:

$$\lambda = 10 \quad [\text{zdarzeń na dzień}].$$

Oznacza to, że czasy między kolejnymi szkodami są niezależne i mają rozkład wykładniczy:

$$T_i \sim \text{Exp}(\lambda),$$

przy czym średni czas między szkodami wynosi $1/\lambda = 0,1$ dnia.

Kwota pojedynczej szkody X_i jest zmienną losową o rozkładzie wykładniczym ze średnią \$1000:

$$E(X_i) = 1000,$$

czyli parametr skali wynosi 1000, a gęstość:

$$f(x) = \frac{1}{1000} e^{-x/1000}, \quad x > 0.$$

W chwili wystąpienia i -tej szkody kapitał ulega natychmiastowej redukcji:

$$U(t_i^+) = U(t_i^-) - X_i.$$

Szukane jest prawdopodobieństwo:

$$p = P(U(t) > 0 \text{ dla wszystkich } t \in [0, 365]).$$

3.3 Metoda zdarzeń dyskretnych

W metodzie zdarzeń dyskretnych symulujemy tylko chwile, w których stan systemu ulega zmianie. W tym modelu są to wyłącznie chwile zgłoszenia szkód.

Algorytm jednej replikacji:

1. Ustaw $t = 0$ oraz $U = 25000$.
2. Wygeneruj odstęp do kolejnej szkody:

$$\Delta \sim \text{Exp}(\lambda).$$

3. Jeżeli $t + \Delta > 365$, dolicz wpłaty do końca roku i zakończ replikację.
4. W przeciwnym razie:

- zwiększ kapitał o wpłaty w okresie $(t, t + \Delta)$:

$$U \leftarrow U + c\Delta;$$

- ustaw $t \leftarrow t + \Delta$;
- wygeneruj kwotę szkody $X \sim \text{Exp}(1000)$;
- zmniejsz kapitał: $U \leftarrow U - X$;
- jeżeli $U \leq 0$, zakończ replikację wynikiem negatywnym.

5. Powtarzaj krok 2.

Ponieważ między szkodami kapitał rośnie liniowo, minimum na każdym takim odcinku występuje bezpośrednio po obsłużeniu szkody. Wystarczy zatem sprawdzać warunek dodatniego kapitału po każdym zdarzeniu szkody.

3.4 Implementacja

Symulację wykonano w języku Python. Poniżej przedstawiono główną funkcję jednej replikacji:

```
def symuluj_jedna_sciezke():
    czas = 0.0
    kapital = 25000.0

    while True:
        odstep = random.expovariate(10.0)
        czas_szkody = czas + odstep

        if czas_szkody > 365.0:
            kapital += 11000.0 * (365.0 - czas)
            return kapital > 0.0

        kapital += 11000.0 * (czas_szkody - czas)
        czas = czas_szkody

        kwota_szkody = random.expovariate(1.0 / 1000.0)
        kapital -= kwota_szkody

    if kapital <= 0.0:
        return False
```

Listing 3: Fragment kodu przedstawiający replikację

Prawdopodobieństwo p estymujemy metodą Monte Carlo:

$$\hat{p} = \frac{1}{N} \sum_{j=1}^N I_j,$$

gdzie $I_j = 1$, jeżeli w j -tej replikacji kapitał pozostał dodatni przez cały okres $[0, 365]$, a w przeciwnym razie $I_j = 0$.

W obliczeniach przyjęto:

$$N = 100000.$$

3.5 Wyniki symulacji

Po wykonaniu $N = 100000$ niezależnych replikacji otrzymano:

Tabela 1: Wyniki estymacji parametrów symulacji

Wielkość	Wartość
Liczba replikacji	100000
Liczba sukcesów	90637
Liczba niepowodzeń	9363
Estymator $\hat{p}$	0,9064
95% przedział ufności	(0,9045; 0,9082)

Oznacza to, że w około 90,64% symulowanych scenariuszy kapitał firmy pozostał dodatni przez cały rok.

Interpretacja wyniku Średnia liczba szkód w ciągu 365 dni wynosi około:

$$\lambda \cdot 365 = 10 \cdot 365 = 3650.$$

Średni łączny koszt szkód:

$$3650 \cdot 1000 = 3\,650\,000.$$

Średni napływ wpłat w ciągu roku:

$$11000 \cdot 365 = 4\,015\,000.$$

Teoretyczna średnia zmiana kapitału (bez uwzględnienia zmienności) jest dodatnia, jednak duża losowość kwot pojedynczych szkód powoduje, że w części scenariuszy następuje bankructwo.

3.6 Odpowiedź końcowa

Metodą zdarzeń dyskretnych oszacowano, że prawdopodobieństwo utrzymania dodatniego kapitału przez pierwsze 365 dni wynosi:

$$\hat{p} \approx 0,9064$$

czyli około:

$$90,6\%.$$

Przy 95% poziomie ufności można przyjąć przedział:

$$(0,9045; 0,9082).$$

Spis rysunków

1	Rozkład par (r_n, r_{n+1}) w teście 2D.	7
2	Rozkład trójek (r_n, r_{n+1}, r_{n+2}) w teście 3D.	8

Spis tabel

1	Wyniki estymacji parametrów symulacji	11
---	---	----

Spis fragmentów kodu

1	Implementacja dwóch rozwiązań zadania	3
2	Implementacja algorytmu kwadratowego von Neumanna	5
3	Fragment kodu przedstawiający replikację	11